Assignment 6

```
Name : Gaurang Tyagi
Roll No. : 16
```

Question

Write a Python script to demonstrate a 3-way handshake synchronization process of the TCP protocol between two nodes. Set the Urgent Flag to ‘1’ and Destination Port to '5394’ in the packet. Also, verify the details of the packet received on the receiver end.

CODE

```
from scapy.all import IP, TCP, Raw, send, sniff, AsyncSniffer
import threading
import time


# ============================================================
# CONFIGURATION
# ============================================================

SERVER_IP = "127.0.0.1"

CLIENT_PORT = 12345
SERVER_PORT = 5394

CLIENT_SEQ = 1000
SERVER_SEQ = 2000

INTERFACE = "lo"
CAPTURE_TIMEOUT = 15


# ============================================================
# RECEIVER
# ============================================================

def receiver():
    print("\n[Receiver] Waiting for SYN packet...")

    # STEP 1: Receive the client's SYN packet.
    syn_packets = sniff(
        filter=f"tcp and dst port {SERVER_PORT}",
        count=1,
        timeout=CAPTURE_TIMEOUT,
        iface=INTERFACE,
```

```
    lfilter=lambda packet: (
        TCP in packet
        and packet[TCP].flags.S
        and not packet[TCP].flags.A
    )
)

if not syn_packets:
    print("[Receiver] SYN packet not received!")
    return

syn_packet = syn_packets[0]
syn_tcp = syn_packet[TCP]

print("\n[Receiver] STEP 1: SYN received")
print("Source Port:", syn_tcp.sport)
print("Destination Port:", syn_tcp.dport)
print("Sequence Number:", syn_tcp.seq)

# STEP 2: Send SYN-ACK to the client.
syn_ack_packet = (
    IP(src=SERVER_IP, dst=SERVER_IP)
    / TCP(
        sport=SERVER_PORT,
        dport=CLIENT_PORT,
        seq=SERVER_SEQ,
        ack=syn_tcp.seq + 1,
        flags="SA"
    )
)

print("\n[Receiver] STEP 2: Sending SYN-ACK")
send(syn_ack_packet, verbose=False)

# STEP 3: Receive the client's final ACK with URG = 1.
ack_packets = sniff(
    filter=f"tcp and dst port {SERVER_PORT}",
    count=1,
    timeout=CAPTURE_TIMEOUT,
    iface=INTERFACE,
    lfilter=lambda packet: (
        TCP in packet
        and packet[TCP].flags.A
        and packet[TCP].flags.U
        and not packet[TCP].flags.S
    )
)

if not ack_packets:
    print("[Receiver] Final ACK with URG flag not received!")
    return

ack_packet = ack_packets[0]
ack_tcp = ack_packet[TCP]

# ============================================================
# VERIFY THE RECEIVED PACKET
# ============================================================
```

```
    print("\n[Receiver] STEP 3: Final ACK received")
    print("\n========== RECEIVED PACKET VERIFICATION ==========")

    print("Source IP:", ack_packet[IP].src)
    print("Destination IP:", ack_packet[IP].dst)
    print("Source Port:", ack_tcp.sport)
    print("Destination Port:", ack_tcp.dport)
    print("Sequence Number:", ack_tcp.seq)
    print("Acknowledgment Number:", ack_tcp.ack)

    print("\nTCP FLAGS:")
    print("SYN:", int(ack_tcp.flags.S))
    print("ACK:", int(ack_tcp.flags.A))
    print("URG:", int(ack_tcp.flags.U))
    print("Urgent Pointer:", ack_tcp.urgptr)

    # Verify the required packet fields.
    checks = {
        "Source Port is 12345": ack_tcp.sport == CLIENT_PORT,
        "Destination Port is 5394": ack_tcp.dport == SERVER_PORT,
        "Sequence Number is 1001": ack_tcp.seq == CLIENT_SEQ + 1,
        "Acknowledgment Number is 2001": (
            ack_tcp.ack == SERVER_SEQ + 1
        ),
        "SYN flag is 0": not ack_tcp.flags.S,
        "ACK flag is 1": bool(ack_tcp.flags.A),
        "URG flag is 1": bool(ack_tcp.flags.U),
        "Urgent Pointer is 1": ack_tcp.urgptr == 1
    }

    print("\n========== VERIFICATION RESULTS ==========")

    for description, passed in checks.items():
        print(f"{description}: {'PASS' if passed else 'FAIL'}")

    print("\nComplete Packet:")
    ack_packet.show()

    if all(checks.values()):
        print("\n[Receiver] Packet verified successfully!")
        print("[Receiver] Three-Way Handshake Demonstration Completed!")
    else:
        print("\n[Receiver] Packet verification failed.")


# ============================================================
# SENDER
# ============================================================

def sender():
    # Give the receiver time to start listening.
    time.sleep(1)

    print("\n[Sender] Initiating TCP Three-Way Handshake")

    # Prepare the asynchronous sniffer before sending SYN.
    # It waits for the server's SYN-ACK packet.
    syn_ack_sniffer = AsyncSniffer(
        filter=f"tcp and dst port {CLIENT_PORT}",
```

```
        count=1,
        iface=INTERFACE,
        lfilter=lambda packet: (
            TCP in packet
            and packet[TCP].flags.S
            and packet[TCP].flags.A
        )
    )

    syn_ack_sniffer.start()
    time.sleep(0.2)

    # STEP 1: Send SYN.
    syn_packet = (
        IP(src=SERVER_IP, dst=SERVER_IP)
        / TCP(
            sport=CLIENT_PORT,
            dport=SERVER_PORT,
            seq=CLIENT_SEQ,
            flags="S"
        )
    )

    print("\n[Sender] STEP 1: Sending SYN")
    send(syn_packet, verbose=False)

    # STEP 2: Wait for the SYN-ACK response.
    syn_ack_sniffer.join(timeout=CAPTURE_TIMEOUT)

    # Stop the sniffer if no matching packet arrived in time.
    if syn_ack_sniffer.running:
        syn_ack_sniffer.stop(join=True)

    syn_ack_packets = syn_ack_sniffer.results or []

    if not syn_ack_packets:
        print("[Sender] SYN-ACK not received!")
        return

    syn_ack_packet = syn_ack_packets[0]
    syn_ack_tcp = syn_ack_packet[TCP]

    print("\n[Sender] STEP 2: SYN-ACK received")
    print("Sequence Number:", syn_ack_tcp.seq)
    print("Acknowledgment Number:", syn_ack_tcp.ack)

    # Verify the SYN-ACK before sending the final ACK.
    if syn_ack_tcp.ack != CLIENT_SEQ + 1:
        print("[Sender] Invalid SYN-ACK acknowledgment number!")
        return

    if not (syn_ack_tcp.flags.S and syn_ack_tcp.flags.A):
        print("[Sender] Invalid SYN-ACK flags!")
        return

    # STEP 3: Send final ACK with URG = 1.
    ack_packet = (
        IP(src=SERVER_IP, dst=SERVER_IP)
        / TCP(
```

```
            sport=CLIENT_PORT,
            dport=SERVER_PORT,
            seq=CLIENT_SEQ + 1,
            ack=syn_ack_tcp.seq + 1,
            flags="AU",
            urgptr=1
        )
        / Raw(load=b"TCP Handshake Completed")
    )

    print("\n[Sender] STEP 3: Sending ACK with URG=1")
    send(ack_packet, verbose=False)

    print("\n[Sender] Final ACK sent successfully!")


# ============================================================
# MAIN PROGRAM
# ============================================================

if __name__ == "__main__":
    receiver_thread = threading.Thread(target=receiver)
    sender_thread = threading.Thread(target=sender)

    # Start receiver first so it is ready to capture the SYN.
    receiver_thread.start()
    sender_thread.start()

    sender_thread.join()
    receiver_thread.join()

    print("\n========== PROGRAM FINISHED ==========")
```

OUTPUT

```
[Receiver] Waiting for SYN packet...

[Sender] Initiating TCP Three-Way Handshake

[Sender] STEP 1: Sending SYN

[Receiver] STEP 1: SYN received
Source Port: 12345
Destination Port: 5394
Sequence Number: 1000

[Receiver] STEP 2: Sending SYN-ACK

[Sender] STEP 2: SYN-ACK received
Sequence Number: 2000
Acknowledgment Number: 1001

[Sender] STEP 3: Sending ACK with URG=1

[Sender] Final ACK sent successfully!

[Receiver] STEP 3: Final ACK received

========== RECEIVED PACKET VERIFICATION ==========
Source IP: 127.0.0.1
Destination IP: 127.0.0.1
Source Port: 12345
Destination Port: 5394
Sequence Number: 1001
Acknowledgment Number: 2001

TCP FLAGS:
SYN: 0
ACK: 1
URG: 1
Urgent Pointer: 1
```

```
========== VERIFICATION RESULTS ==========
Source Port is 12345: PASS
Destination Port is 5394: PASS
Sequence Number is 1001: PASS
Acknowledgment Number is 2001: PASS
SYN flag is 0: PASS
ACK flag is 1: PASS
URG flag is 1: PASS
Urgent Pointer is 1: PASS

Complete Packet:
###[ Ethernet ]###
  dst       = 00:00:00:00:00:00
  src       = 00:00:00:00:00:00
  type      = IPv4
###[ IP ]###
     version   = 4
     ihl       = 5
     tos       = 0x0
     len       = 63
     id        = 1
     flags     =
     frag      = 0
     ttl       = 64
     proto     = tcp
     chksum    = 0x7cb6
     src       = 127.0.0.1
     dst       = 127.0.0.1
     \options   \
###[ TCP ]###
        sport     = 12345
        dport     = 5394
        seq       = 1001
        ack       = 2001
        dataofs   = 5
        reserved  = 0
        flags     = AU
        window    = 8192
        chksum    = 0xb5cc
```

```
      \options   \
 ###[ TCP ]###
         sport     = 12345
         dport     = 5394
         seq       = 1001
         ack       = 2001
         dataofs   = 5
         reserved  = 0
         flags     = AU
         window    = 8192
         chksum    = 0xb5cc
         urgptr    = 1
         options   = []
 ###[ Raw ]###
            load      = b'TCP Handshake Completed'


 [Receiver] Packet verified successfully!
 [Receiver] Three-Way Handshake Demonstration Completed!

 ========== PROGRAM FINISHED ==========
```